

`stylometric-transfer` is an interpretable stylometric profiling and author-style transfer system built around explicit JSON fingerprints, local measurements, and controlled LLM orchestration.

The repository is designed to help users model their own writing style and apply it to new text while preserving meaning and keeping every style decision auditable.

Positioning

This project is not a fine-tuning framework and not an opaque embedding pipeline. It is an explicit style-modeling stack:

- Local stylometric measurements are computed from user corpora.
- A structured, human-readable fingerprint JSON is produced.
- Rewriting is constrained by that fingerprint plus deterministic post-processing.
- Deviations are reported when constraints conflict with source meaning or formatting.

End-to-end capabilities

- Corpus fingerprinting from archives (`.zip` , `.tar*`) and common text formats (`.txt` , `.md` , `.rst` , `.html` , `.docx`).
- Rich measurement layer including sentence/paragraph distributions, punctuation rates, function-word signals, lexical preferences/avoidance, stance, rhetoric, cadence, syntax texture, and repetition features.
- Voice-aware filtering that excludes non-author regions from profiling and rewrite control (blockquotes, references, footnotes, inline citations, boilerplate notices).
- Fiction/non-fiction quote handling with automatic detection and manual overrides.
- Proper-name and phrase-noise suppression via deterministic filters plus optional LLM phrase validation/ranking.
- Fingerprint generation with chunk/merge support when prompt size exceeds configured token budgets.
- Style application with strict meaning-preservation goals, structured deviations, and optional style-retry loops using local compliance scoring.
- Humanizer-guideline integration with conflict filtering, mandatory normalization controls, heading-case management, and locale spelling enforcement (LLM and deterministic layers).
- Runtime variability controls via bounded "perplexity" presets, seeded variance, and optional model roster cycling.
- HTML fingerprint visualization for inspection and review.

Local API capabilities

The repo includes a local HTTP API (`fingerprint_api.py`) intended for non-production use:

- `POST /make` : create and store a fingerprint from text and return a GUID.
- `POST /apply` : apply a stored fingerprint to new text.
- `POST /rate` : estimate stylometric match probability between text and a stored fingerprint.
- `POST /similarity` : compare two stored fingerprints with component-level diagnostics.
- `GET /health` , `GET /openapi.yaml` , `GET /openapi.json` for operations and docs.

API server port can be set via `--port` or `--api` (alias). Fingerprints are tracked in the repository-root `fingerprint_store/` directory using GUID-indexed JSON artifacts.

GUI harness and operator UX

The repo includes a GUI demo/test harness (`fingerprint_api_harness.py`) so users can exercise all API endpoints without curl. A portable launcher script (`scripts/fingerprint_api_harness.sh`) sets a default `API_PORT` constant and can be run from anywhere, with runtime overrides still supported.

Key interfaces and artifacts

- CLI tools: `fingerprint_style.py` , `apply_fingerprint.py` , `show_fingerprint.py` , `fingerprint_api.py` , `fingerprint_api_harness.py`
- Wrapper scripts: `scripts/*.sh` for portable invocation
- Shared modules: `common.py` , `utils.py`
- Config modalities shared across CLI/API usage: `config.llm.json` , `config.tunables.json` , `config.avoid.txt` , `config.local_spelling_rules.json` , `config.llm.roster.json`
- API specs: `api/swagger/openapi.yaml` , `api/swagger/openapi.json`
- Technical rationale: `Research-Paper.md`

Intended use and constraints

The system is optimized for personal writing consistency, explicit control, and explainability. It emphasizes inspectable style transfer and reproducibility, not covert impersonation or black-box imitation.